MECHATRONICS AND MEMS

Subject Code : 23A03702d

UNIT IV : Microprocessors, Microcontrollers and PLCs

Syllabus: Architecture of Microprocessor, Microcontroller and Programmable Logic Controller, PLC Programming using ladder diagrams, logics, latching, sequencing, timers relays and counters, data handling, Analog input/output, selection of controllers.

1. Controllers in Mechatronics

- The **controller** is the brain of a mechatronic system. It reads the inputs (sensors), runs a program / logic, and gives outputs to actuators.
- Common controllers: **Microprocessor (uP) based systems, Microcontroller (uC), Programmable Logic Controller (PLC)** , also DSP, FPGA and industrial PC.
- Number systems used: binary, hexadecimal. Data bus width = 8, 16, 32 bits.

2. Architecture of a Microprocessor

- A **microprocessor** is a single chip CPU (VLSI). It needs **external memory and I/O chips** to make a complete system - so it is general purpose and flexible. Eg: Intel 8085 (8 bit), 8086 (16 bit), Pentium.

Main blocks of the microprocessor

- **ALU (Arithmetic Logic Unit):** does arithmetic (add, subtract) and

logic (AND, OR, XOR, compare) operations.

- **Registers:** fast internal storage - **accumulator** (result of ALU), general purpose registers (B, C, D, E, H, L), **program counter PC** (address of next instruction), **stack pointer SP** , instruction register, flag register (zero, carry, sign, parity).
- **Control unit:** fetches, decodes and executes instructions; generates timing and control signals.
- **Buses:**
  - **Address bus** (unidirectional) - carries the memory/IO address. 16 lines can access 2^16 = 64 KB.
  - **Data bus** (bidirectional) - carries the data (8 bit).
  - **Control bus** - signals such as RD, WR, ALE, IO/M, interrupt (INTR, TRAP).

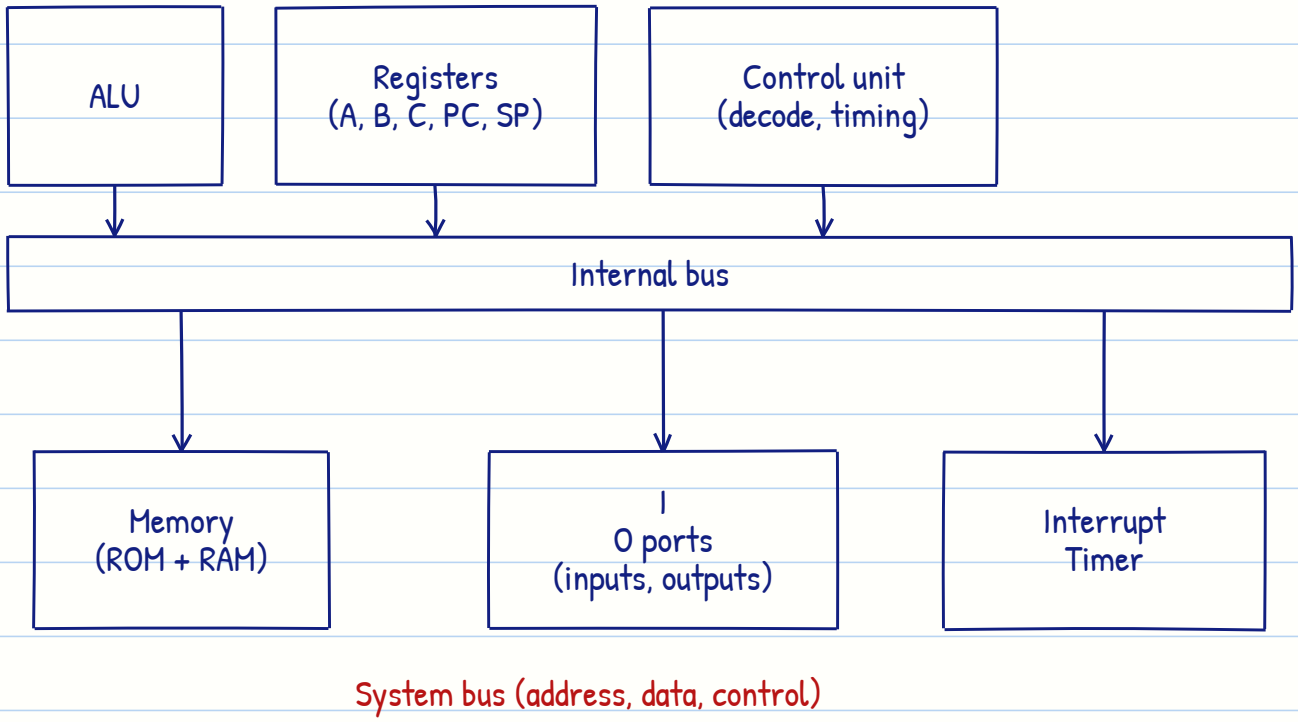


Fig: Microprocessor based system - CPU with external memory and I/O

Instruction cycle (fetch – decode – execute)

- **Fetch:** PC puts the address on the address bus, instruction is read from memory into the instruction register, PC = PC + 1. **Decode:** control unit decodes the opcode. **Execute:** ALU / registers / memory perform the operation. Then the next fetch.
- 8085 features: 8 bit data, 16 bit address, 5 V single supply, clock up to 3 MHz, 74 instructions, 5 interrupts, 40 pin.

Von Neumann vs Harvard architecture

- **Von Neumann:** single memory and bus for program and data (microprocessors, PC). **Harvard:** separate program and data memories and buses – faster, used in microcontrollers (PIC, AVR, 8051) and DSP.

RISC and CISC

- **CISC:** many complex instructions (x86). **RISC:** small set of simple instructions, one cycle each, pipelining (ARM, AVR, PIC).

3. Architecture of a Microcontroller

- A **microcontroller** is a **computer on a single chip** : CPU + RAM + ROM/Flash + I/O ports + timers/counters + serial port + interrupt + ADC etc. all in one IC. It is **dedicated to one control task** (embedded system). Eg: Intel 8051, PIC16F877, AVR ATmega328 (Arduino), ARM Cortex, ESP32.

Block diagram (8051 as example)

- **CPU (8 bit) with ALU, accumulator A, B register, PSW, PC, DPTR, SP.**
- **ROM / Flash:** 4 KB on-chip program memory. **RAM:** 128 bytes data

memory (register banks, bit addressable area, scratch pad).

- **I/O ports:** 4 ports P0, P1, P2, P3 (32 I/O lines). P0 and P2 also work as multiplexed address/data bus when external memory is used.
- **Timers/counters:** two 16 bit (T0, T1) - for delay, counting events, baud rate. **Serial port:** UART full duplex. **Interrupts:** 5 sources (2 external, 2 timers, 1 serial). Oscillator / clock circuit (11.0592 MHz).

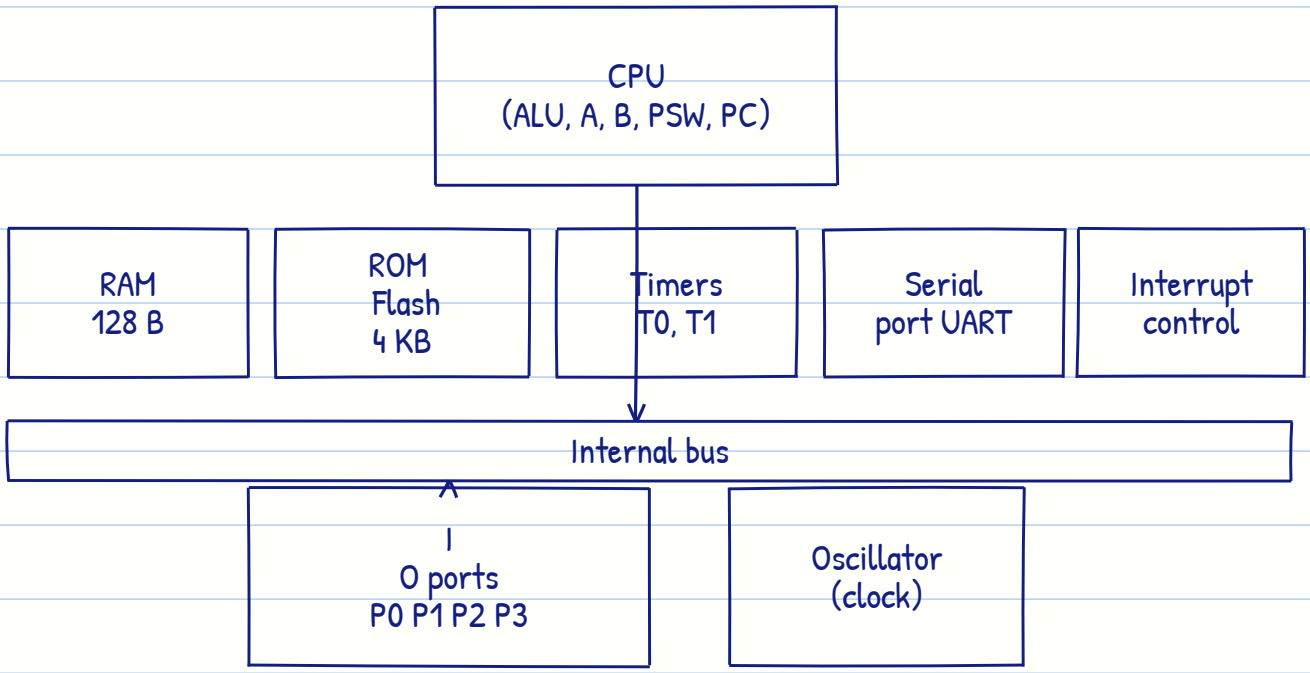


Fig: Block diagram of 8051 microcontroller

Microprocessor vs Microcontroller

Point	Microprocessor	Microcontroller
Chip contents	Only CPU	CPU + RAM + ROM + I/O + timer
External parts	Needs memory, I/O chips	Very few (all inside)
Cost and size	Higher, bigger board	Low, compact
Power	High	Low

Speed	High (GHz)	Lower (MHz)
Use	General purpose (computer)	Dedicated (washing machine, car ECU)
Architecture	Von Neumann mostly	Harvard mostly
Bit manipulation	Few instructions	Many bit-level instructions

Applications of microcontrollers in mechatronics

- Motor speed / position control, CNC and robot joints, automobile ECU (engine, ABS), washing machine, microwave oven, remote controls, smart meters, 3D printer, drones, IoT nodes.

4. Programmable Logic Controller (PLC)

- **Definition (NEMA):** A PLC is a digital electronic device with **programmable memory** to store instructions that implement functions like logic, sequencing, timing, counting and arithmetic to control machines and processes through digital or analog inputs and outputs.
- Designed for the **harsh industrial environment** - dust, noise, vibration, EMI, temperature. First used in 1968 by General Motors to replace relay panels (Modicon 084, Dick Morley).
- Eg: Siemens S7, Allen-Bradley (Rockwell), Mitsubishi FX/Q, Omron, Delta, ABB.

Advantages over relay logic

- Changes by software, no rewiring; compact; reliable (no contacts to wear); fast; easy fault finding (indicator LEDs, diagnostics); counting, timing, arithmetic and communication; low cost per function; program can be stored and copied.

Architecture of a PLC

- **Central Processing Unit (CPU):** microprocessor that executes the user program in a repeated **scan cycle** ; has the system program (operating system).
- **Memory: ROM** (system program), **RAM** (user program, data, I/O image table; battery backup), **EEPROM / Flash** (permanent program storage).
- **Input module:** takes field signals (push buttons, limit switches, sensors) 24 V DC / 230 V AC, with **opto isolation** , filter and level shifting to logic levels.
- **Output module:** gives power to loads (contactor, solenoid valve, lamps, motor starters). Types: **relay** (AC/DC, slow, high current), **transistor** (DC, fast), **triac** (AC, fast).
- **Power supply:** converts 230 V AC to 24 V DC / 5 V for the CPU and modules.
- **Programming device:** PC with software / handheld programmer. **HMI/SCADA** and communication ports (RS232, RS485, Ethernet, Profibus, Modbus).

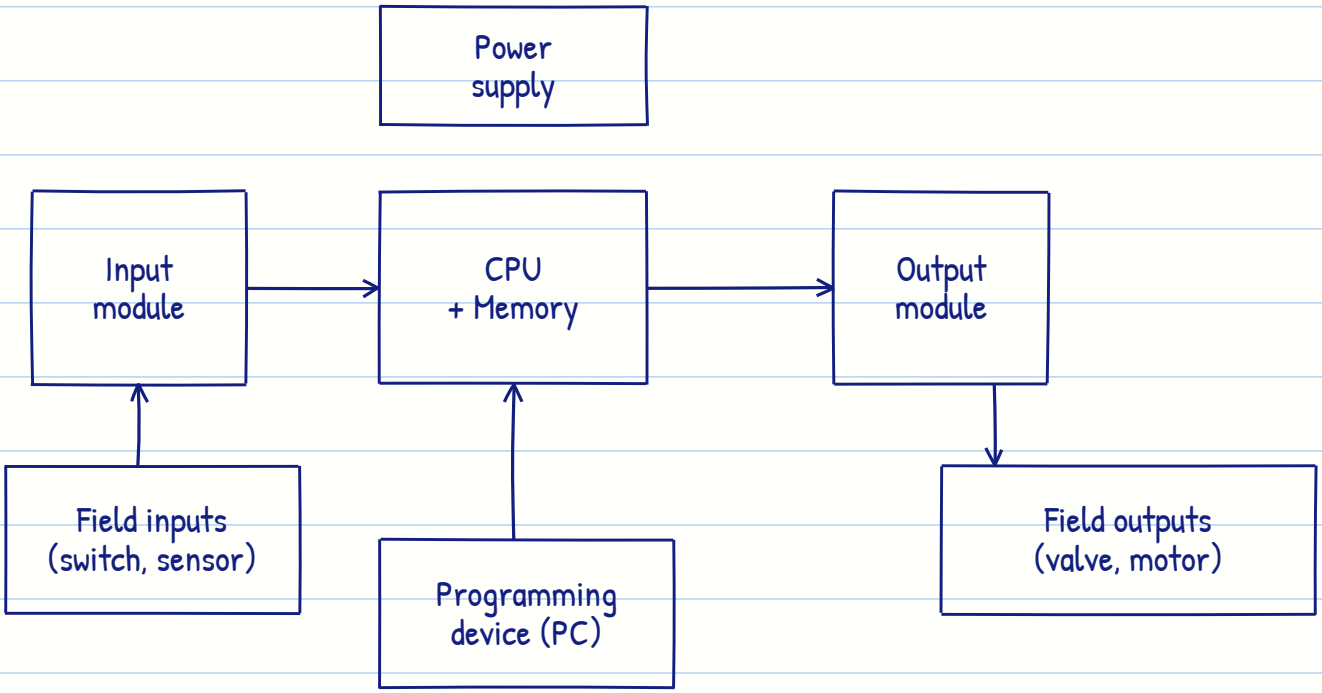


Fig: Architecture of a PLC

Scan cycle of a PLC

- **1. Read inputs** (copy the input states to the input image table) -> **2. Execute the program** (rung by rung, left to right, top to bottom) -> **3. Update outputs** (write the output image table to output terminals) -> **4. Housekeeping / communication, self diagnosis** . Then repeat. Scan time is typically 1 to 100 ms.

Types of PLC by size

- **Compact / fixed (nano, micro):** all in one box, up to 128 I/O - small machines. **Modular:** rack, CPU and plug in modules - **large** , more than 1000s of I/O, expandable.

Input / Output devices

- Inputs: push buttons, selector switches, limit switches, proximity and photo sensors, pressure switches. Outputs: relay, contactors, solenoid valves, indicator lamps, buzzers, motor drives.

5. PLC Programming Languages (IEC 61131-3)

- **Ladder Diagram (LD)** - graphical, like relay circuit; most popular. **Function Block Diagram (FBD)** - blocks. **Structured Text (ST)** - high level like Pascal. **Instruction List (IL)** - assembly like mnemonics (LD, AND, OR, OUT). **Sequential Function Chart (SFC)** - steps and transitions.

6. PLC Programming using Ladder Diagrams

- A ladder diagram has two vertical **power rails** (left = live, right = neutral/return) and horizontal **rungs** between them, looking like a ladder. Power "flows" left to right when the logic of a rung is TRUE (**power flow**). Each rung has **inputs (conditions) on the left** and **one output (coil) on the right.**

Symbols

- **Normally Open contact (NO) -| |-:** True when the input is ON (1). **Normally Closed contact (NC) -|/|-:** True when the input is OFF (0). **Output coil -( )-** . **Set / Reset coil** (latch/unlatch). Addresses: I0.0, X0 (input), Q0.0, Y0 (output), M (memory), T (timer), C (counter).

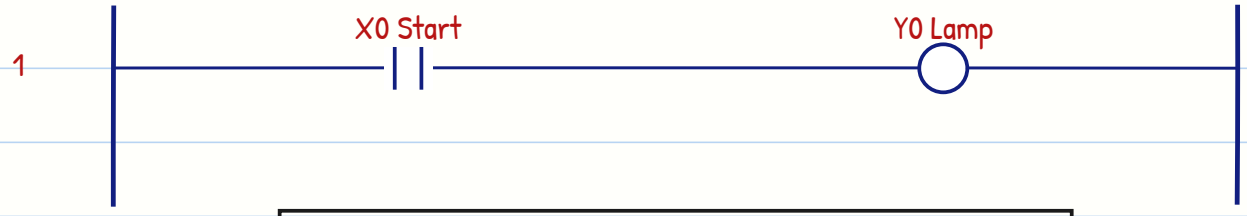


Fig: Rung 1 - simple ON action: Y0 = X0

6.1 Logic functions in ladder

- **AND:** contacts in **series** - output ON only when all the contacts are closed. **OR:** contacts in **parallel** - output ON if any contact closed.

NOT: use NC contact. **NAND, NOR, XOR** are obtained by combining these.

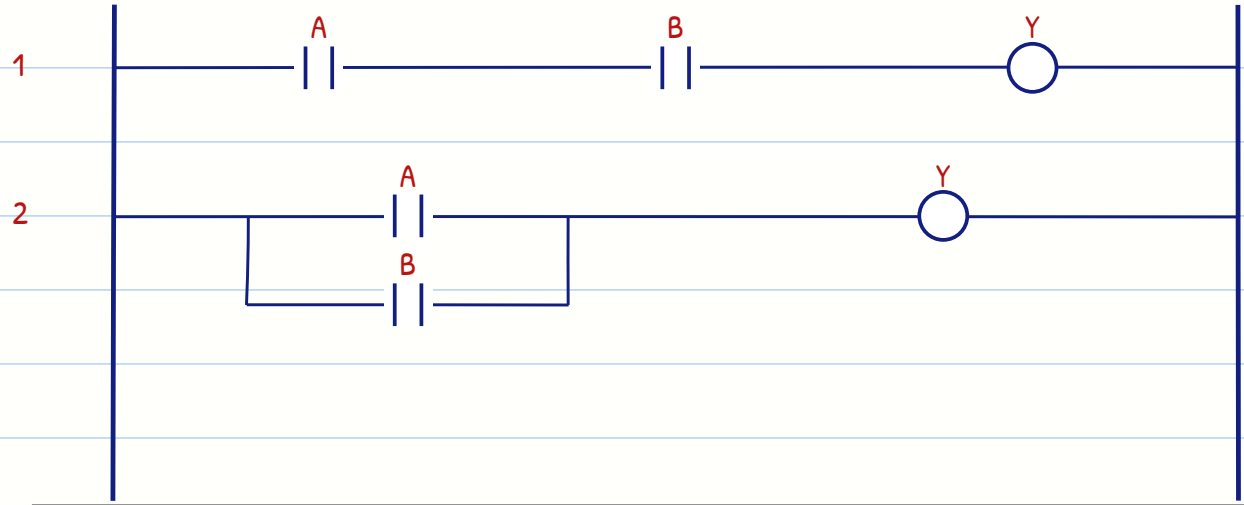


Fig: AND (series) and OR (parallel) logic - rung 1 AND, rung 2 OR

- **XOR:** Y = A B' + A' B -> two parallel branches: (A NO in series with B NC) OR (A NC in series with B NO).
- **NOT:** NC contact. **NAND:** parallel of NC contacts. **NOR:** series of NC contacts.

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

6.2 Latching (self-holding / sealing circuit)

- A push button gives only a momentary signal. To keep the output ON after the button is released, a **holding (sealing) contact** of the output itself is placed in parallel with the start button. **Stop button (NC)** is in series to break the circuit.
- Y0 = (Start + Y0) . Stop'. Used for motor start/stop control (DOL

starter), lamp control, alarms.

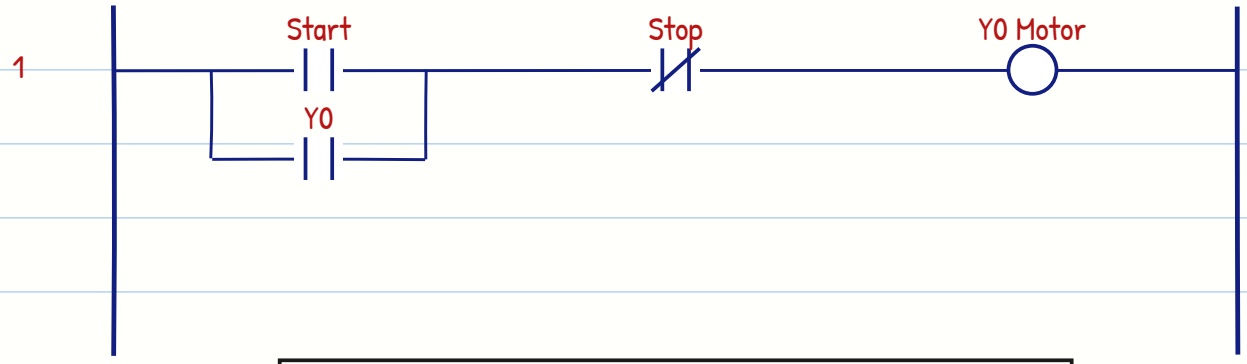


Fig: Latching circuit - motor start / stop

- **SET / RESET (latch / unlatch) instructions:** SET coil makes the output ON and it stays ON even if the input goes OFF until RESET coil clears it. Priority: the last executed instruction wins (reset dominant is safer).

6.3 Interlocking

- Prevent two outputs from being ON together (eg forward and reverse contactors): NC contact of the other output is put in series in each rung.

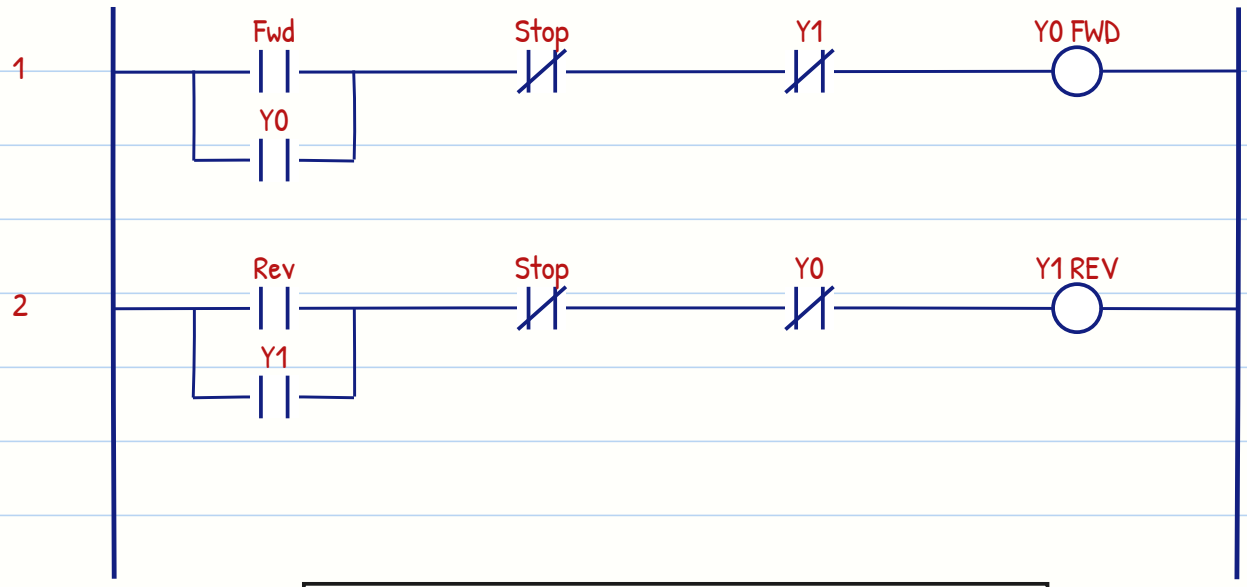


Fig: Forward / Reverse interlock circuit

6.4 Sequencing

- Operations done in a fixed order: step 1 then 2 then 3. Done by using **output of previous step (or its memory bit) as a condition for the next** and the end signal (limit switch, timer) to move on, and the next step **resets** the previous step.
- Eg: Cylinder sequence A+ B+ A- B-: Step 1 A+ ON when Start; limit switch a1 -> Step 2 B+; b1 -> Step 3 A-; a0 -> Step 4 B-; b0 -> end/ready for next cycle. Each step is latched; the next one unlatches the earlier one. Can also use **step counters / shift register / SFC** .

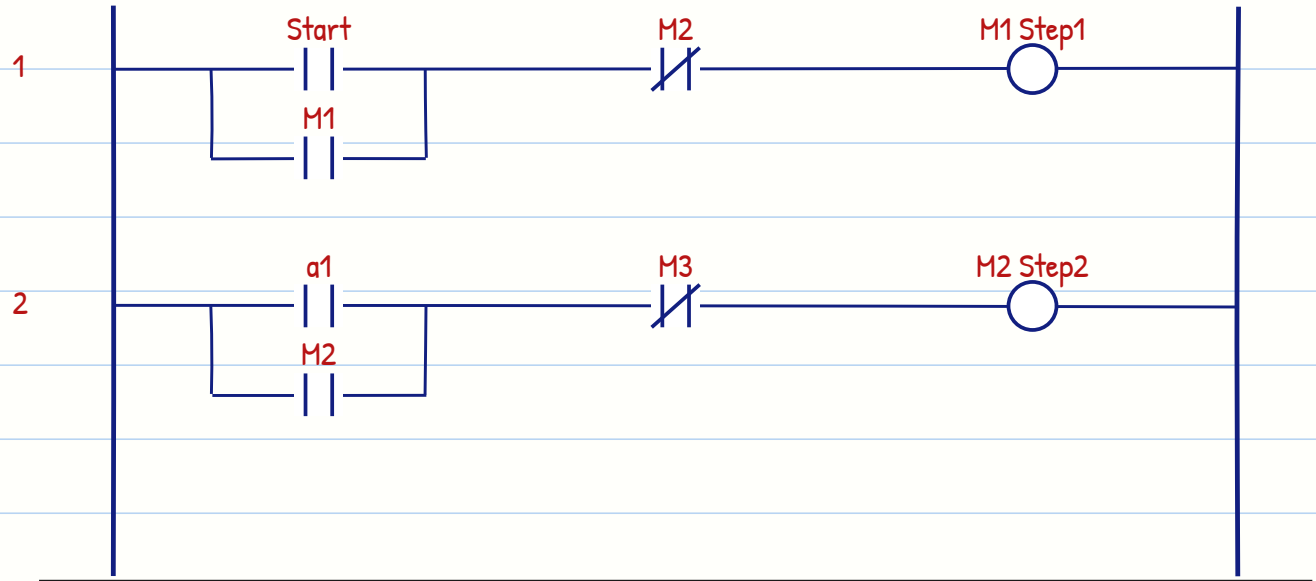


Fig: Sequencing - step 1 enables step 2 which then resets step 1 (M2 interlocks)

7. Timers

- Timers give a time delay. Each has a **preset value (PT), accumulated value (ET), enable input and output bit (Q/DN).** Time base 1 ms, 10 ms, 100 ms or 1 s.
- **TON (On-delay timer):** output turns ON after the preset time **once the input stays ON** ; when the input goes OFF the output resets

immediately. Used for delayed start, motor start sequencing.

- **TOF (Off-delay timer):** output ON immediately with the input; stays ON for the preset time after the input goes OFF. Used for cooling fans and lights.
- **TP (pulse timer):** output pulse of fixed length. **RTO (Retentive timer)** keeps accumulated time on loss of input; needs a reset.

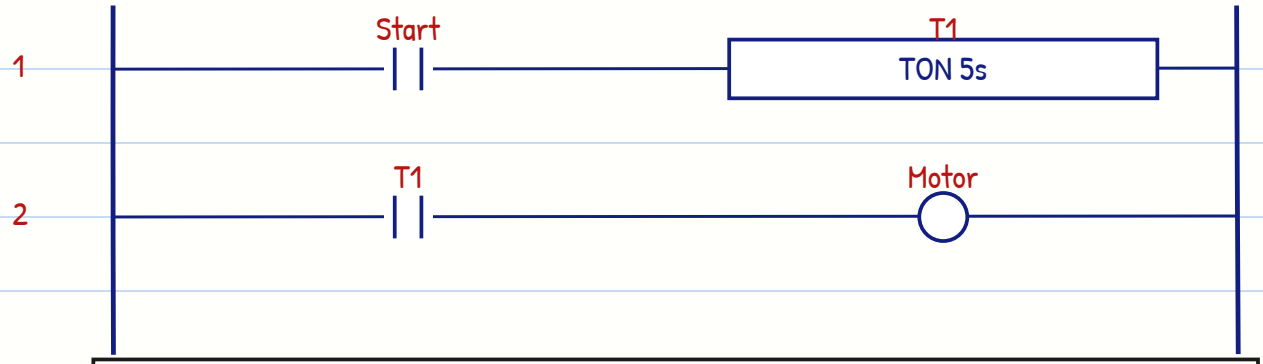


Fig: ON delay timer - Motor starts 5 s after Start is pressed

Timing diagram of TON

- Input goes high at t0 -> timer counts -> at t0 + PT output goes high -> input low -> output low immediately.

Cyclic (flasher) circuit

- Two timers: T1 (on time) and T2 (off time) connected so each one resets the other - lamp blinks. Used in alarms.

8. Relays (control relays) in PLC

- **Internal (auxiliary) relays (M bits / flags)** are software relays - no physical existence - used for storing intermediate logic results, latching, and sequencing.
- **Master control relay (MCR)** switches OFF all the rungs in a zone together (emergency stop). **Jump / Label, subroutine** instructions skip

parts of program.

- **External relays and contactors** are driven by PLC outputs to isolate and switch heavy loads, eg a 24 V DC output driving a 230 V contactor coil.

9. Counters

- Count **events** - parts on a conveyor, bottles, machine cycles. Have **preset value (PV), current value (CV), count input, reset input, and output (Q)** . Counts on the **rising edge** of the input.
- **CTU (Count Up):** CV increases by 1 for each pulse; output ON when CV >= PV. **CTD (Count Down):** CV decreases from PV; output ON when CV = 0. **CTUD (Up-Down counter)** - both.
- **RESET** clears the count to 0 (or loads PV for CTD). Counters are retentive (keep count on power loss in many PLCs).
- **Cascading counters** (Counter output used as the count pulse of the next) for larger counts. **Timer + counter** also gives long time delays.

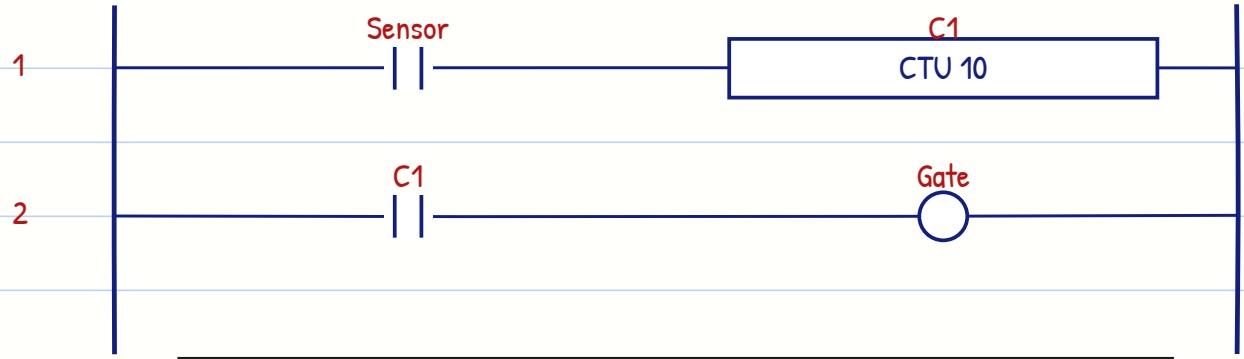


Fig: Counter - gate operates after counting 10 parts

- Eg: Counting 10 bottles on a conveyor: photoelectric sensor counts the bottles -> after 10 the conveyor stops and the packing cylinder operates; the counter is reset.

10. Data Handling in PLC

- PLCs also handle data (not only ON / OFF bits): **bit (1), byte (8), word (16 bit), double word (32 bit)** and floating point (REAL).
- **Data transfer / move instructions:** MOV, copy - load a constant or value of a register to another.
- **Arithmetic instructions:** ADD, SUB, MUL, DIV, increment / decrement, square root.
- **Comparison instructions:** equal (EQ), not equal, greater than (GT), less than (LT), GE, LE - used in rungs as contacts, eg: temperature > 80 -> fan ON.
- **Logical and bit operations:** AND, OR, XOR, NOT on words, shift and rotate (shift register), conversion (BCD to binary).
- **Data registers (D, DB):** store variable values such as set points, preset values, recipes, production counts. **Table / array and sequencer instructions.**
- **Data communication:** with HMI, SCADA, drives and other PLCs by Ethernet / Profibus / Modbus / RS485.

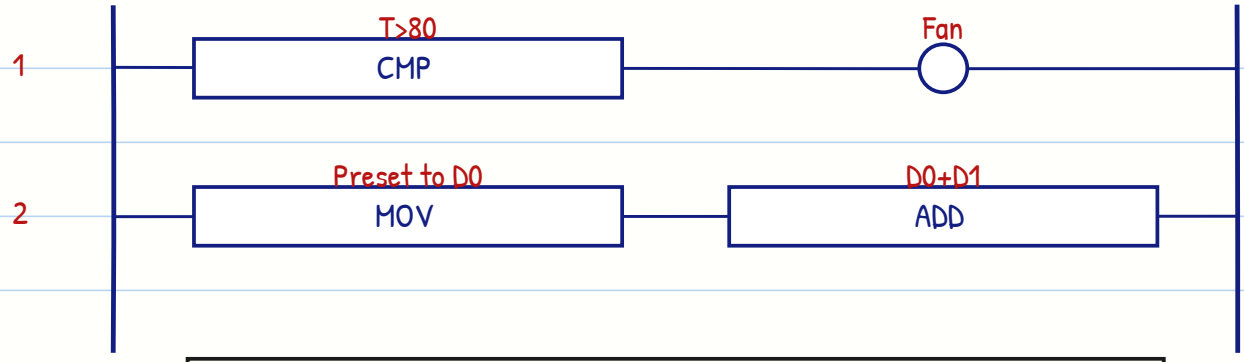


Fig: Data handling - compare and arithmetic rungs

II. Analog Input / Output

- Real world quantities (temperature, pressure, level, speed) are **analog (continuous)** . A PLC works with digital numbers, so **analog modules** are used.

Analog Input (AI) module

- Gets signals **0-10 V, +/-10 V, 0-20 mA or 4-20 mA** from transmitters / sensors (RTD, thermocouple modules). An **ADC (Analog to Digital Converter)** converts it to a digital number (12 bit: 0 - 4095, 16 bit: 0-65535 or 0 - 27648 in Siemens). Resolution = Full scale / 2^n.
- **4-20 mA** is preferred - low noise, long distance, and 4 mA (live zero) shows a wire break.
- **Scaling:** Engineering value = (Count / Max count) x (EUmax - EUmin) + EUmin. Eg: 0-10 bar sensor on 12 bit ADC: count 2048 = 5 bar.

Analog Output (AO) module

- **DAC (Digital to Analog Converter)** changes a digital number to 0-10 V or 4-20 mA to drive **VFD speed reference, proportional valve, control valve positioner, servo.**

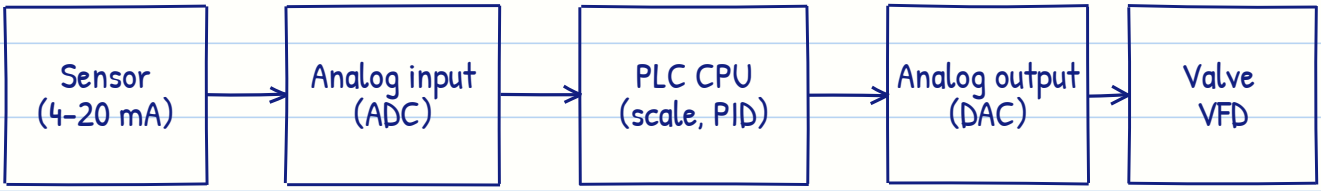


Fig: Analog signal path in a PLC (PID control loop)

- **PID function block:** CPU compares the set point and the process value (analog input) and sends the control output (analog output) - temperature / pressure / flow loops.

12. Selection of Controllers

- **Number and type of I/O** (digital, analog, special - high speed counter, pulse train for stepper/servo) and **20 - 30% spare I/O** for expansion.
- **Speed (scan time) and processing power;** memory size for program and data.
- **Complexity of the control task** (simple logic, PID, motion, data handling); **programming languages** supported.
- **Environment:** temperature, humidity, vibration, EMI, hazardous place, protection class (IP).
- **Communication** (Ethernet, Profibus, Modbus, OPC), HMI/SCADA support.
- **Cost, availability of spares and service,** familiarity of the staff, software and training cost; vendor support.
- **Reliability, redundancy and safety (SIL) requirement;** power supply; size.

Point	Microprocessor	Microcontroller	PLC
Use	General computing	Embedded product	Industrial control
Environment	Office / lab	Product inside	Harsh factory floor
Programming	Assembly, C, high level	Assembly, C, Embedded C	Ladder, FBD, ST

I/O	External chips	Built in, small	Modular, rugged, isolated
Cost	High for system	Lowest	High
Best for	PC, complex tasks	Mass produced, small units	Automation of machines / plants

Important Questions (Unit IV)

- Draw and explain the architecture of 8085 microprocessor / 8051 microcontroller.
- Compare microprocessor and microcontroller.
- Explain the architecture of a PLC with a block diagram. Write the scan cycle.
- Write ladder diagrams for AND, OR, latching and sequencing (A+ B+ A- B-).
- Explain timers (TON, TOF) and counters (CTU, CTD) in a PLC with examples.
- Explain data handling and analog input / output in PLC. Write the selection criteria for controllers.